```v
`timescale 1ns / 1ps

module top1(
    input  wire        clk_100m,   // 开发板 100MHz 原始时钟
    input  wire        rst,        // 复位信号
    output wire [10:0] led,        // 11个 LED (表1: led[10:0])
    output wire [6:0]  seg,        // 数码管段码 (来自 display.v)
    output wire [7:0]  an          // 数码管位选 (来自 display.v)
);

    wire        lclk;              // 1Hz 慢时钟
    wire [31:0] pc_out;            // 当前指令地址
    wire [31:0] pc_next;           // 下一条指令地址 (PC+4)
    wire        inst_ce;           // 指令存储器使能信号
    wire [31:0] inst;              // 指令存储器输出的 32 位指令内容

    // 控制器输出的控制信号
    wire        memtoreg, memwrite, alusrc, regdst, regwrite, jump, branch;
    wire [2:0]  alucontrol;
    wire        pcsrc;             // 对应 led[2]

    assign pc_next = pc_out + 32'h4;

    // 目前实验仅为取指译码，pcsrc 逻辑暂时设为 0（代表始终顺序执行）
    assign pcsrc = 1'b0;

    // ============================================
    // 3. 模块实例化与连接
    // ============================================

    // (A) 时钟分频器: 100Mhz -> 1hz
    clk_div u_clk_div (
        .hclk(clk_100m),
        .lclk(lclk)
    );

    // (B) PC 寄存器: 存储当前地址
    // 注意: 输入是 pc_next (即 pc_out + 4)
    pc u_pc (
        .clk(lclk),
        .rst(rst),
        .npc(pc_next),       // 输入: 下一条地址
        .pc(pc_out),         // 输出: 当前地址
        .inst_ce(inst_ce)
```

```
    );


    blk_mem_gen_0 u_ram (
        .clka(lclk),
        .ena(inst_ce),
        .wea(1'b0),
        .addra(pc_out[9:2]),
        .dina(32'h0),
        .douta(inst)
    );


//    blk_mem_gen_0 your_instance_name (
//  .clka(clka),    // input wire clka
//  .ena(ena),      // input wire ena
//  .wea(wea),      // input wire [0 : 0] wea
//  .addra(addra),  // input wire [7 : 0] addra
//  .dina(dina),    // input wire [31 : 0] dina
//  .douta(douta)  // output wire [31 : 0] douta
//);


    // (D) 控制器 (包装了你的 maindec 和 aludec)
    controller u_controller (
        .op(inst[31:26]),
        .funct(inst[5:0]),
        .memtoreg(memtoreg),
        .memwrite(memwrite),
        .branch(branch),
        .alusrc(alusrc),
        .regdst(regdst),
        .regwrite(regwrite),
        .jump(jump),
        .alucontrol(alucontrol)
    );

    display u_display (
        .clk(clk_100m),
        .reset(rst),
        .s(inst),         // 数码管显示当前取出的 32 位指令
        .seg(seg),
        .ans(an)
    );
```

```
    assign led[0]    = memtoreg;
    assign led[1]    = memwrite;
    assign led[2]    = pcsrc;
    assign led[3]    = alusrc;
    assign led[4]    = regdst;
    assign led[5]    = regwrite;
    assign led[6]    = jump;
    assign led[7]    = branch;
    assign led[10:8] = alucontrol;

endmodule
```

```
//clk.div
`timescale 1ns / 1ps

module clk_div(
    input  wire hclk,    // 100MHz 高频时钟
    output reg  lclk     // 1Hz 慢时钟
);

    // 定义计数器，需要能容纳 100,000,000 这个数字 (至少 27 位)
    reg [31:0] cnt;

    initial begin
        cnt = 32'd0;
        lclk = 1'b0;
    end

    always @(posedge hclk) begin
        // 当计数到 50,000,000 - 1 时翻转信号，实现 1Hz
        if (cnt == 32'd49_999_999) begin
            cnt <= 32'd0;
            lclk <= ~lclk;
        end else begin
            cnt <= cnt + 1'b1;
        end
    end

endmodule
```

```
`timescale 1ns / 1ps
```

```
module pc(
    input wire clk,rst,npc,
    output reg pc,inst_ce
    );
    wire [31:0] npc;
    reg [31:0] pc;
    always @(posedge clk) begin
        if (rst) begin
        pc <= 32'h0;
        inst_ce <= 0;
        end
        else begin
        pc <= npc;
        inst_ce <= 1;
        end
    end

endmodule
```

```
// ctrlor
`timescale 1ns / 1ps

module controller(
    input  wire [5:0] op,           // 对应 inst[31:26]
    input  wire [5:0] funct,        // 对应 inst[5:0]
    output wire       memtoreg,
    output wire       memwrite,
    output wire       branch,
    output wire       alusrc,
    output wire       regdst,
    output wire       regwrite,
    output wire       jump,
    output wire [2:0] alucontrol
);


    wire [1:0] aluop_wire;

    maindec u_maindec (
        .op(op),
        .aluop(aluop_wire),         // 输出连到内部导线
        .memtoreg(memtoreg),        // 其余直接连到 controller 的输出端口
        .memwrite(memwrite),
```

```
        .branch(branch),
        .alusrc(alusrc),
        .regdst(regdst),
        .regwrite(regwrite),
        .jump(jump)
    );

    // 2. 实例化 ALU 译码器 (aludec)
    aludec u_aludec (
        .funct(funct),
        .aluop(aluop_wire),          // 接收来自 maindec 的内部导线信号
        .aluctrl(alucontrol)         // 将你 aludec 里命名的 aluctrl 连到顶层的
alucontrol 输出
    );

endmodule
```

```
// minidec
`timescale 1ns / 1ps

module maindec(
    input wire [5:0] op,
    output reg [1:0] aluop,
    output reg        memtoreg, memwrite,
    output reg        branch, alusrc,
    output reg        regdst, regwrite,
    output reg        jump
    );
   always @(*) begin
        case(op)
            6'b000000: begin // R-type 指令 (例如 add, sub, and, or, slt)
                regwrite = 1'b1; regdst = 1'b1; alusrc = 1'b0; branch =
1'b0;
                memwrite = 1'b0; memtoreg = 1'b0; jump = 1'b0; aluop =
2'b10;
            end

            6'b100011: begin // lw 指令 (Load word)
                regwrite = 1'b1; regdst = 1'b0; alusrc = 1'b1; branch =
1'b0;
                memwrite = 1'b0; memtoreg = 1'b1; jump = 1'b0; aluop =
2'b00;
            end
```

```
            6'b101011: begin // sw 指令 (Store word)
                // 表格中 regdst 和 memtoreg 是 X (无关项)，在代码中统一赋 0 以
免产生锁存器
                regwrite = 1'b0; regdst = 1'b0; alusrc = 1'b1; branch =
1'b0;
                memwrite = 1'b1; memtoreg = 1'b0; jump = 1'b0; aluop =
2'b00;
            end

            6'b000100: begin // beq 指令 (Branch on equal)
                // 表格中 regdst 和 memtoreg 是 X (无关项)，统一赋 0
                regwrite = 1'b0; regdst = 1'b0; alusrc = 1'b0; branch =
1'b1;
                memwrite = 1'b0; memtoreg = 1'b0; jump = 1'b0; aluop =
2'b01;
            end

            6'b001000: begin // addi 指令 (Add immediate)
                regwrite = 1'b1; regdst = 1'b0; alusrc = 1'b1; branch =
1'b0;
                memwrite = 1'b0; memtoreg = 1'b0; jump = 1'b0; aluop =
2'b00;
            end

            6'b000010: begin // j 指令 (Jump)
                // 表格中很多是 X (无关项)，统一赋 0，jump 信号单独置 1
                regwrite = 1'b0; regdst = 1'b0; alusrc = 1'b0; branch =
1'b0;
                memwrite = 1'b0; memtoreg = 1'b0; jump = 1'b1; aluop =
2'b00;
            end

            default: begin   // 默认情况 (避免产生 Latch)
                regwrite = 1'b0; regdst = 1'b0; alusrc = 1'b0; branch =
1'b0;
                memwrite = 1'b0; memtoreg = 1'b0; jump = 1'b0; aluop =
2'b00;
            end
        endcase
    end
endmodule
```

```
// aludec
`timescale 1ns / 1ps
```

```
module aludec(
    input wire [5:0] funct,
    input wire [1:0] aluop,
    output reg [2:0] aluctrl
    );
    always @(*) begin
        case(aluop)
        2'b00:begin
            aluctrl = 3'b010;
        end
        2'b01:begin
            aluctrl = 3'b110;
        end
        2'b10:begin
            case(funct)
                6'b100000:begin
                    aluctrl = 3'b010;
                end
                6'b100010:begin
                    aluctrl = 3'b110;
                end
                6'b100100:begin
                    aluctrl = 3'b000;
                end
                6'b100101:begin
                    aluctrl = 3'b001;
                end
                6'b101010:begin
                    aluctrl = 3'b111;
                end
             default:begin
                    aluctrl = 3'b011;
                    end
            endcase
        end
        default:begin
        aluctrl = 3'b011;
        end
        endcase
    end
endmodule
```

```
`timescale 1ns / 1ps

module test_bench(

    );

    reg rst;
    reg clk;
    wire [7:0] ans;
    wire [6:0] seg;
    wire [10:0] led;
    initial
    begin
        clk = 1'b0;
        rst = 1'b1;
        #500;
        rst = 1'b0;
    end
    always #10 clk = ~clk;
    top1 top1(
        .clk_100m(clk),
        .rst(rst),
        .seg(seg),
        .an(ans),
        .led(led)
        );
endmodule
```